

ECE0202 Lecture 18

Fixed-point and Floating-point Arithmetic

中文逐页详细讲义

根据课程 PPT 整理

本讲主线：这节课讲两个问题。第一，**fixed-point number (定点数)**：把小数变成整数来算，适合没有 FPU 或追求速度/确定性的嵌入式系统。第二，**floating-point number (浮点数)**：用 IEEE 754 格式表示范围很大的实数，适合科学计算，但有舍入、溢出、特殊值等问题。

你需要抓住的核心：定点数的“小数点位置”由格式固定；浮点数的“小数点位置”由 exponent (指数) 决定，所以它能表示更大范围，但精度分布不均匀。

目录

1	Slide 1: 标题页	2
2	Slide 2: Topics - 本讲内容	3
3	Slide 3: Fixed-point Numbers - 定点数基本例子	4
4	Slide 4: Unsigned Fixed-point Representation - UQm.n	5
5	Slide 5: 把 3.141593 转成 UQ4.12	6
6	Slide 6: 练习 - 把 $e \approx 2.71828$ 转成 UQ4.12	7
7	Slide 7: Signed Fixed-point Representation - Qm.n	8
8	Slide 8: 把 -3.141593 转成 Q3.12	9
9	Slide 9: 练习 - 把 -2.71828 转成 Q3.12	10
10	Slides 10-11: Fixed-point Addition and Subtraction - 定点加减法	11
11	Slides 12-13: Fixed-point Multiplication - 定点乘法推导	12
12	Slide 14: UQ16.16 乘法代码 - UMULL	13
13	Slide 15: Q15.16 乘法代码 - SMULL	14
14	Slides 16-17: Fixed-point Division - 定点除法	15
15	Slide 18: Floating-Point Numbers - 浮点数引入	16
16	Slide 19: Normalization - 规格化	17

17 Slide 20: IEEE Std 754 单精度公式	18
18 Slides 21-24: 解码 0xC1FF0000	19
19 Slide 25: 把 14.5 编码成 IEEE 754 单精度	20
20 Slide 26: 练习 - 把 0.375 编码成 IEEE 754 单精度	21
21 Slide 27: Special Values - 特殊值	22
22 Slides 28-29: Overflow and Underflow - 溢出与下溢	23
23 Slide 30: Subnormalized Float Number - 非规格化数	24
24 Slide 31: Resolution - 分辨率	25
25 Slide 32: Rounding - 舍入规则	26
26 Slide 33 / 原 PPT 38: FPU Registers - 浮点寄存器	27
27 Slide 34 / 原 PPT 39: FPU Instructions - 浮点算术指令	28
28 Slide 35 / 原 PPT 40: FPU Instructions - 比较、转换、移动	29
29 Slide 36 / 原 PPT 41: FPU Instructions - Load/Store 浮点寄存器	30
30 Lecture 18 总结	31

Slide 1: 标题页

这一页只是说明 Lecture 18 的主题: Fixed-point and Floating-point Arithmetic。

关键词:

- **Fixed-point number (定点数)**: 小数点位置固定。例如 UQ4.12 表示整数部分 4 bit, 小数部分 12 bit。硬件里面本质仍然存整数。
- **Floating-point number (浮点数)**: 小数点位置可以“浮动”。它用 sign (符号位)、exponent (指数)、fraction (小数部分) 来表示数。
- **Arithmetic (算术运算)**: 加、减、乘、除。这里关心的是 CPU/寄存器如何表示小数并做运算。

第一性原理: 计算机底层只存 0 和 1。所谓“小数”, 不是天然存在的东西, 而是程序员和硬件共同约定的解释方式。定点数和浮点数就是两种不同的解释规则。

Slide 2: Topics - 本讲内容

PPT 列出两个主题：

1. Fixed-point Numbers (定点数)
2. Floating-Point Number (浮点数)

为什么嵌入式系统要学它？

- 传感器经常产生小数，例如温度 23.75°C、电压 1.234 V、距离 12.6 cm。
- Cortex-M4 可能带 FPU (Floating-Point Unit, 浮点运算单元)，但不是所有 MCU 都有，也不是所有场景都该用浮点。
- 定点数速度快、结果更可控；浮点数范围大、写代码方便，但要理解误差。

本讲学习目标：你要会把小数转换成定点格式，会理解 IEEE 754 单精度浮点数的 sign/exponent/fraction，会解释 overflow、underflow、NaN、rounding 等概念。

Slide 3: Fixed-point Numbers - 定点数基本例子

PPT 给出二进制定点数：

$$01101.101_2$$

它的意思是：小数点左边按 $2^0, 2^1, 2^2, \dots$ 计算；小数点右边按 $2^{-1}, 2^{-2}, 2^{-3}, \dots$ 计算。

$$01101.101_2 = 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-3} = 13.625$$

逐位解释：

- 左边 $01101_2 = 13_{10}$ 。
- 右边 $.101_2 = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 0.5 + 0.125 = 0.625$ 。
- 所以总值是 $13 + 0.625 = 13.625$ 。

直白理解：定点数就是“二进制小数点固定在那里”。硬件里不一定真的存一个小数点，小数点位置由格式约定。

Slide 4: Unsigned Fixed-point Representation - UQm.n

UQm.n 是 unsigned fixed-point format (无符号定点格式)。

- U: Unsigned, 无符号, 只能表示非负数。
- Q: Quantization format, 量化格式。工程里常用 Q 格式表示定点数。
- m: 整数部分 bit 数。
- n: 小数部分 bit 数。

PPT 公式:

$$f = A_h + A_l \times 2^{-n}$$

其中:

- A_h : 整数部分。
- A_l : 小数部分对应的整数。
- 2^{-n} : 因为小数部分有 n 位, 所以要缩小 2^n 倍。

PPT 例子:

$$10101.101_2 = 21 + 5 \times 2^{-3} = 21.625$$

为什么 $A_l = 5$? 因为小数部分是 $101_2 = 5_{10}$ 。小数部分有 3 位, 所以要乘 $2^{-3} = 1/8$, 即 $5/8 = 0.625$ 。

易错点: UQm.n 的小数点不是动态的。UQ4.12 就永远是 4 位整数、12 位小数。你不能临时改变小数点位置。

Slide 5: 把 3.141593 转成 UQ4.12

目标：把十进制小数 $f = 3.141593$ 转成 UQ4.12。

UQ4.12 的含义：

- 4 bits integer part (4 位整数部分)。
- 12 bits fraction part (12 位小数部分)。
- 总共 16 bits。
- 因为小数有 12 位，所以真实值和整数编码之间差一个 2^{12} 的比例。

$$I = \text{round}(f \times 2^{12})$$

代入：

$$3.141593 \times 4096 = 12867.964928$$

四舍五入：

$$I = 12868$$

转成二进制：

$$12868 = 0011\ 0010\ 0100\ 0100_2$$

所以 UQ4.12 写成：

$$0011.0010\ 0100\ 0100_2 = 0x3244$$

为什么会有误差？ 因为 12 位小数无法表示所有十进制小数，只能表示 $1/4096$ 的整数倍。3.141593 不是 $1/4096$ 的精确整数倍，所以必须 round (舍入)。

定点转换口诀： 十进制实数 f 转 Q/UQ 格式，先乘 2^n ，再四舍五入成整数，最后按二进制/十六进制存储。

Slide 6: 练习 - 把 $e \approx 2.71828$ 转成 UQ4.12

PPT 要求: Convert e to unsigned fixed-point UQ4.12 format, write in hex.

步骤:

$$I = \text{round}(2.71828 \times 2^{12})$$

$$2.71828 \times 4096 = 11134.07488$$

$$I = 11134$$

把 11134 转成十六进制:

$$11134_{10} = 0x2B7E$$

所以:

$$e \approx 0x2B7E \text{ in UQ4.12}$$

检查一下:

$$\frac{11134}{4096} = 2.71826171875$$

和 2.71828 很接近, 误差约为:

$$2.71826171875 - 2.71828 \approx -1.828125 \times 10^{-5}$$

考试写法: 写清楚 $\times 2^{12}$ 、round、转 hex 三步即可。不要只写答案, 否则过程分容易丢。

Slide 7: Signed Fixed-point Representation - Qm.n

Qm.n 是 signed fixed-point format (有符号定点格式)。

和 UQm.n 的区别:

- UQm.n 没有符号位, 只能表示非负数。
- Qm.n 有 1 个 sign bit (符号位), 可以表示正数和负数。
- 总 bit 数是 $N = m + n + 1$ 。

PPT 公式:

$$A = -1 \times b_{N-1} \times 2^{N-1} + \sum_{i=0}^{N-2} b_i 2^i$$
$$f = \frac{A}{2^n}$$

这个公式在说什么?

- 先把整串 bit 当作一个 two's complement integer (补码整数) 得到 A 。
- 再除以 2^n , 得到真实定点小数 f 。

直白说:

$$\text{定点真实值} = \frac{\text{补码整数值}}{2^n}$$

重点: 有符号定点数不是单独发明一套负数规则, 它直接使用 two's complement (补码)。小数点只是在补码整数上人为规定的位置。

Slide 8: 把 -3.141593 转成 Q3.12

目标: Convert $f = -3.141593$ to signed fixed-point Q3.12.

Q3.12 的含义:

- 1 bit sign.
- 3 bits integer part.
- 12 bits fraction part.
- 总共 $1 + 3 + 12 = 16$ bits.

计算整数编码:

$$I = \text{round}(-3.141593 \times 2^{12}) = -12868$$

因为是 signed format, 要用 16-bit two's complement 表示 -12868 。先写 $+12868$:

$$12868 = 0011\ 0010\ 0100\ 0100_2$$

取补码得到:

$$1100\ 1101\ 1011\ 1100_2 = 0xCDBC$$

所以:

$$-3.141593 \approx 0xCDBC \text{ in Q3.12}$$

为什么要补码? 因为 ARM 和大多数现代处理器用 two's complement 表示有符号整数。Q3.12 只是把这个补码整数除以 2^{12} 来解释成小数。

Slide 9: 练习 - 把 -2.71828 转成 Q3.12

PPT 要求: Convert -2.71828 to signed fixed-point Q3.12, write in hex.

步骤:

$$I = \text{round}(-2.71828 \times 4096)$$

$$-2.71828 \times 4096 = -11134.07488$$

$$I = -11134$$

16-bit two's complement:

$$65536 - 11134 = 54402 = 0xD482$$

所以:

$$-2.71828 \approx 0xD482 \quad \text{in Q3.12}$$

检查:

$$\frac{-11134}{4096} = -2.71826171875$$

误差约为:

$$-2.71826171875 - (-2.71828) = 1.828125 \times 10^{-5}$$

易错点: 负数定点转换不是“前面加负号”那么简单。最后存进寄存器/内存的是补码 bit pattern。

Slides 10-11: Fixed-point Addition and Subtraction - 定点加减法

这两页讲定点数加减为什么可以直接用整数加减指令。

假设格式是 UQ16.16 或 Q15.16。真实值和整数编码关系是：

$$f_A = I_A \times 2^{-16}, \quad f_B = I_B \times 2^{-16}$$

如果：

$$f_C = f_A + f_B$$

那么：

$$f_C = (I_A + I_B) \times 2^{-16}$$

所以：

$$I_C = I_A + I_B$$

关键结论：只要两个定点数格式相同，加减法可以直接对底层整数编码做加减。

汇编代码：

```
; r0 = fixed-point number fa
; r1 = fixed-point number fb
; r2 = fixed-point number fc

ADD r2, r0, r1    ; fc = fa + fb
SUB r2, r0, r1    ; fc = fa - fb
```

为什么可以这么简单？因为 I_A 、 I_B 都默认带有同样的比例因子 2^{-16} 。相同单位相加，比例因子不变。

前提：两个数必须是同一种 Q 格式。例如 Q15.16 和 Q15.16 可以直接加；Q3.12 和 Q15.16 不能直接加，必须先统一格式。

Slides 12-13: Fixed-point Multiplication - 定点乘法推导

定点乘法比加法麻烦，因为比例因子会相乘。

假设：

$$f_A = I_A \times 2^{-16}, \quad f_B = I_B \times 2^{-16}$$

乘法：

$$f_C = f_A \times f_B = I_A I_B \times 2^{-32}$$

但我们希望结果仍然是 Q16.16 或 Q15.16：

$$f_C = I_C \times 2^{-16}$$

所以：

$$I_C = I_A \times I_B \times 2^{-16} = \frac{I_A \times I_B}{2^{16}}$$

直白理解：

- 两个 Q16.16 数相乘，小数位数会变成 32 位。
- 但结果还要回到 16 位小数，所以要右移 16 位。
- 乘法可能产生 64-bit 结果，所以不能只看低 32 位。

易错点：定点乘法不是直接 ‘MUL’ 完就结束。你必须处理 “多出来的 16 个小数位”，通常通过右移 16 位实现。

Slide 14: UQ16.16 乘法代码 - UMULL

这页讲无符号定点乘法。UQ16.16 是 unsigned，所以使用 **UMULL**。

UMULL = Unsigned Multiply Long，无符号长乘法。

它把两个 32-bit 无符号整数相乘，产生 64-bit 结果：

$$\text{result} = r3 : r2$$

其中：

- $r2$ 保存低 32 位。
- $r3$ 保存高 32 位。

代码：

```
UMULL r2, r3, r0, r1    ; r3:r2 = r0 * r1
LSLS  r3, r3, #16       ; high word left shift 16
LSRS  r2, r2, #16       ; low word right shift 16
ORR   r4, r2, r3        ; combine two parts
```

为什么要这样移位？

- 乘完后得到 64-bit 结果。
- 定点公式要求除以 2^{16} ，也就是整体右移 16 位。
- 64-bit 右移 16 位后，目标结果的低 32 位由原来的高低两个 word 拼出来。

一句话：UQ16.16 乘法：先用 UMULL 得到 64-bit 乘积，再整体右移 16 位，取回 32-bit 定点结果。

Slide 15: Q15.16 乘法代码 - SMULL

这页和上一页几乎一样，但格式换成 Q15.16，是 signed fixed-point，所以要用 **SMULL**。

SMULL = Signed Multiply Long，有符号长乘法。

代码：

```
SMULL r2, r3, r0, r1    ; signed 64-bit product, r3:r2 = r0 * r1
LSLS  r3, r3, #16      ; shift high word left
LSRS  r2, r2, #16      ; shift low word right
ORR   r4, r2, r3        ; pack result
```

为什么 signed 和 unsigned 要分开？

- 对于正数，UMULL 和 SMULL 结果可能一样。
- 对于负数，bit pattern 是补码。如果你用 unsigned 方式解释负数，结果会完全错。
- Q15.16 允许负数，所以必须用 signed multiply。

易错点：Q 格式有符号就用 SMULL，无符号就用 UMULL。不要混用。硬件不会替你理解“这其实是负数”。

Slides 16-17: Fixed-point Division - 定点除法

定点除法的核心公式：

$$f_C = f_A \div f_B$$

代入：

$$f_A = I_A 2^{-16}, \quad f_B = I_B 2^{-16}$$

则：

$$f_C = \frac{I_A 2^{-16}}{I_B 2^{-16}} = \frac{I_A}{I_B}$$

但结果要继续用 Q16.16/Q15.16 存储：

$$f_C = I_C 2^{-16}$$

所以：

$$I_C = \frac{I_A \times 2^{16}}{I_B}$$

直白理解：

- 直接 I_A/I_B 会丢掉小数精度。
- 所以先把 I_A 左移 16 位，保留小数位空间。
- 再除以 I_B ，得到仍然带 16 位小数的结果。

代码逻辑：

```
ASRS r1, r0, #31      ; sign extend A to upper word
LSLS r1, r1, #16      ; put sign bits in upper half word
ORR  r1, r1, r0, LSR #16
LSLS r0, r0, #16      ; [r1:r0] = A << 16 with sign extension
ASRS r3, r2, #31      ; [r3:r2] = sign extended B
BL   division_64_bits ; divide 64-bit A by 64-bit/sign-extended B
MOV  r4, r0           ; result in r0
```

为什么麻烦？ ARM 常规 32-bit 除法指令不能直接做完整的 64-bit 除法，而定点除法为了保留小数精度需要把被除数扩大到 64-bit。

Slide 18: Floating-Point Numbers - 浮点数引入

这一页引用 von Neumann 的话：如果你不知道自己在说什么，精确没有意义。
这句话放在这里的意思是：

- 浮点数看起来“很精确”，但它本质上是有限 bit 的近似表示。
- 很多十进制小数在二进制中无法精确表示。
- 你必须知道精度、舍入、溢出、NaN 等问题，否则浮点结果会误导你。

浮点数为什么需要？

- 定点数范围有限。小数点固定后，整数范围和小数精度互相抢 bit。
- 浮点数通过 exponent（指数）移动小数点，能表示非常大和非常小的数。
- 代价是：不是所有数都能精确表示，而且不同数值范围内分辨率不同。

定点 vs 浮点：定点数像一把固定刻度尺；浮点数像可变倍率的显微镜。浮点范围大，但不要以为它无限精确。

Slide 19: Normalization - 规格化

Normalization (规格化) 是浮点数的核心步骤。

IEEE 754 浮点数通常把非零数写成：

$$\pm 1.xxxxx_2 \times 2^e$$

也就是说，二进制小数点左边必须是 1。这一位叫 **hidden bit (隐藏位)**。

PPT 例子：

$$10.746 \times 2^6$$

把 10.746 改成 1.34325×2^3 ，所以：

$$10.746 \times 2^6 = 1.34325 \times 2^9$$

为什么要规格化？

- 让同一个数只有一种标准写法。
- 小数点左边固定为 1 后，这个 1 可以不存，节省 1 bit。
- 这就是 IEEE 754 的 hidden 1。

易错点：规格化不是改变数值，而是改变写法。 10.746×2^6 和 1.34325×2^9 是同一个数。

Slide 20: IEEE Std 754 单精度公式

IEEE 754 single precision (单精度) 用 32 bits 表示一个 float:

- 1 bit sign (符号位)。
- 8 bits exponent (指数域)。
- 23 bits fraction (小数域/尾数小数部分)。

PPT 公式:

$$(-1)^S \times (1 + Fraction) \times 2^{Exponent - Bias}$$

字段解释:

- S : sign bit。0 表示正数, 1 表示负数。
- $Fraction$: 小数部分, 不包括隐藏的 leading 1。
- $Exponent$: 存储在 exponent field 里的无符号整数。
- $Bias$: 偏置值。single precision 的 bias 是 127, double precision 的 bias 是 1023。

为什么需要 Bias? 指数可能为负, 例如 2^{-3} 。但 exponent field 本身是无符号 bit pattern。用 bias 可以把真实指数平移成非负数来存储:

$$Stored\ Exponent = True\ Exponent + 127$$

单精度三件套: sign 决定正负; exponent 决定数量级; fraction 决定有效数字。

Slides 21-24: 解码 0xC1FF0000

这几页是完整解码 IEEE 754 单精度数的例子。

给定：

$$0xC1FF0000$$

转成 32-bit 二进制后拆成：

- Sign = 1。
- Exponent = $10000011_2 = 131$ 。
- Fraction = 0.1111111_2 。

第一步：符号

$$(-1)^S = (-1)^1 = -1$$

所以结果是负数。

第二步：指数

$$\text{True Exponent} = 131 - 127 = 4$$

第三步：fraction

$$0.1111111_2 = 2^{-1} + 2^{-2} + \dots + 2^{-7} = 0.9921875$$

第四步：代入公式

$$\begin{aligned} f &= (-1)^1 \times (1 + 0.9921875) \times 2^4 \\ f &= -1 \times 1.9921875 \times 16 = -31.875 \end{aligned}$$

$$0xC1FF0000 \Rightarrow -31.875$$

易错点：fraction field 存的是 $0.xxxx$ ，但公式里用的是 $1 + \text{Fraction}$ 。这个 1 是 hidden bit，不在 fraction field 里显式存储。

Slide 25: 把 14.5 编码成 IEEE 754 单精度

目标: Encode 14.5 into IEEE 754 single precision.

第一步: 转二进制并规格化

$$14.5 = 1110.1_2$$

移动小数点:

$$1110.1_2 = 1.1101_2 \times 2^3$$

所以:

$$14.5 = (1 + 0.8125) \times 2^3$$

第二步: sign

$$S = 0$$

因为是正数。

第三步: exponent

$$\text{Stored Exponent} = 3 + 127 = 130 = 10000010_2$$

第四步: fraction 规格化后是 1.1101_2 , hidden 1 不存, 所以 fraction field 是:

$$110100000000000000000000$$

最终 32-bit:

$$\begin{aligned} &0\ 10000010\ 110100000000000000000000 \\ &= 01000001011010000000000000000000_2 = 0x41680000 \end{aligned}$$

编码浮点数步骤: 先规格化, 再求 sign, 再加 bias 得 exponent, 最后填 fraction。

Slide 26: 练习 - 把 0.375 编码成 IEEE 754 单精度

目标: Encode 0.375 into IEEE 754 single precision.

第一步: 转二进制

$$0.375 = 0.25 + 0.125 = 2^{-2} + 2^{-3} = 0.011_2$$

第二步: 规格化

$$0.011_2 = 1.1_2 \times 2^{-2}$$

第三步: sign

$$S = 0$$

第四步: exponent

$$\text{Stored Exponent} = -2 + 127 = 125 = 01111101_2$$

第五步: fraction 规格化后 1.1_2 , hidden 1 不存, fraction 是:

$$10000000000000000000000000000000$$

最终:

$$\begin{aligned} & 0 \ 01111101 \ 100000000000000000000000 \\ & = 00111110110000000000000000000000_2 = 0x3EC00000 \end{aligned}$$

$$0.375 \Rightarrow 0x3EC00000$$

Slide 27: Special Values - 特殊值

IEEE 754 保留了一些 exponent pattern 表示特殊值。

特殊值包括：

- **+0 / -0**：正零和负零。exponent 全 0，fraction 全 0，sign 可为 0 或 1。
- **+Infinity / -Infinity（正无穷/负无穷）**：exponent 全 1，fraction 全 0。
- **NaN（Not a Number，不是一个数）**：exponent 全 1，fraction 非 0。

NaN 什么时候出现？

- $0.0/0.0$
- $\sqrt{-1.0}$
- $\log(-10.0)$
- $-\infty + \infty$

QNaN 和 SNaN：

- **QNaN = Quiet NaN，安静 NaN**：通常不会立刻触发异常，只是在后续计算中传播。
- **SNaN = Signaling NaN，信号 NaN**：用于提示异常，可能触发硬件 exception。

易错点：NaN 不等于无穷大。无穷大表示结果超出范围；NaN 表示运算本身无效。

Slides 28-29: Overflow and Underflow - 溢出与下溢

Overflow (溢出): 数太大, float 表示不了。

Underflow (下溢): 数太接近 0, 小到 normalized float 表示不了。

单精度 normalized float 的最小正数:

$$(1 + 0) \times 2^{1-127} = 2^{-126} \approx 1.18 \times 10^{-38}$$

单精度 normalized float 的最大有限数:

$$(2 - 2^{-23}) \times 2^{127} \approx 3.40 \times 10^{38}$$

范围直观理解:

- 大于约 3.40×10^{38} , 会 overflow, 通常变成 infinity。
- 小于约 1.18×10^{-38} 但不等于 0, 进入 subnormal 区域; 再小可能变成 0。

浮点不是无限范围。它只是范围比定点大很多。超过范围照样出问题。

Slide 30: Subnormalized Float Number - 非规格化数

Subnormal number (非规格化数, 也叫 denormalized number) 用来表示 0 和最小 normalized number 之间的非常小的数。

Normalized format:

$$(-1)^S \times (1 + Fraction) \times 2^{Exponent-127}$$

Subnormal format:

$$(-1)^S \times Fraction \times 2^{-126}$$

关键区别:

- Normalized 有 hidden 1, 即 $1 + Fraction$ 。
- Subnormal 没有 hidden 1, 只是 $Fraction$ 。

为什么需要 subnormal?

- 如果没有 subnormal, 最小正 normalized 数和 0 之间会有巨大断层。
- Subnormal 让数值可以逐渐接近 0, 称为 gradual underflow (渐进下溢)。

易错点: Exponent 全 0 时, 不能再套 normalized 公式。此时没有 hidden 1。

Slide 31: Resolution - 分辨率

Resolution (分辨率) 指两个相邻可表示数之间的间隔。

这页用一个假想 5-bit 浮点系统解释：

- 1 bit sign。
- 2 bits exponent。
- 2 bits fraction。

浮点分辨率的核心特点：

- 数值靠近 0 时，相邻数之间间隔小。
- 数值很大时，相邻数之间间隔大。
- 所以浮点数不是“所有地方一样精细”。

这和定点数不同：

- 定点数小数位固定，所以分辨率固定，例如 Q15.16 的最小步长是 2^{-16} 。
- 浮点数指数会变，所以分辨率随数值大小变化。

一句话：浮点数的精度不是绝对均匀的，而是相对精度较稳定；数字越大，间隔也越大。

Slide 32: Rounding - 舍入规则

IEEE 754 定义四种常见 rounding (舍入) 规则。

规则	含义
Round to nearest	舍入到最近的可表示值。默认最常用。
Toward zero / truncation	向 0 舍入，也就是直接截断。正数变小，负数变大。
Toward $+\infty$	向正无穷舍入，也叫 rounding up。结果不小于原数。
Toward $-\infty$	向负无穷舍入，也叫 rounding down。结果不大于原数。

PPT 例子：只允许小数点后 5 位。

- $+0.123456$ nearest 得 $+0.12346$ 。
- $+0.123456$ truncate 得 $+0.12345$ 。
- -0.123456 rounding up 得 -0.12345 ，因为它向 $+\infty$ 靠近。
- -0.123456 rounding down 得 -0.12346 ，因为它向 $-\infty$ 靠近。

易错点：rounding up 不是“绝对值变大”，而是“数轴上向 $+\infty$ 方向”。对负数，rounding up 会让它变得没那么负。

Slide 33 / 原 PPT 38: FPU Registers - 浮点寄存器

FPU = Floating-Point Unit, 浮点运算单元。

Cortex-M4 可带 FPU。FPU 有自己的浮点寄存器，不直接使用普通的 R0-R15 来做浮点计算。

常见寄存器命名：

- **S0-S31**: single-precision registers, 32 个单精度寄存器, 每个 32-bit。
- **D0-D15**: double-precision register view, 两个 S 寄存器合起来可以看成是一个 64-bit D 寄存器。
- 例如 D0 可以由 S0 和 S1 组成。

caller-saved / callee-saved:

- caller-saved: 调用者自己负责保存。
- callee-saved: 被调用函数如果要改它, 必须先保存, 返回前恢复。

这和普通寄存器 R0-R12 的调用约定类似, 只是对象变成浮点寄存器。

FPSCR: Floating-Point Status and Control Register, 浮点状态和控制寄存器。它记录浮点状态, 例如 rounding mode (舍入模式)、exception flags (异常标志)。

直白理解: FPU 就像 CPU 旁边的“小数专用计算器”。它有自己的寄存器和状态位。

Slide 34 / 原 PPT 39: FPU Instructions - 浮点算术指令

这一页列出 FPU 的基本 arithmetic instructions (算术指令)。

常见指令：

- **VADD**: floating-point add, 浮点加法。
- **VSUB**: floating-point subtract, 浮点减法。
- **VMUL**: floating-point multiply, 浮点乘法。
- **VDIV**: floating-point divide, 浮点除法。
- **VABS**: absolute value, 绝对值。
- **VNEG**: negate, 取负。
- **VSQRT**: square root, 开平方。

示例：

```
VADD.F32 S0, S1, S2    ; S0 = S1 + S2, single precision
VMUL.F32 S3, S4, S5    ; S3 = S4 * S5
```

F32 是什么？

- F32 表示 32-bit float, 即 single precision。
- F64 表示 64-bit double, 即 double precision。

注意：有 FPU 不代表你不用理解 IEEE 754。FPU 只是帮你执行规则；舍入、NaN、overflow 这些问题仍然存在。

Slide 35 / 原 PPT 40: FPU Instructions - 比较、转换、移动

这一页列出更多 FPU 指令，重点不是背全表，而是理解类别。

1. Compare (比较)

- **VCMP**: 比较两个浮点数。
- 比较结果会进入 FPSCR，必要时可以转到 APSR 的 NZCV flags，用于条件跳转。

2. Convert (类型转换)

- **VCVT**: convert, 转换。
- 可以 float 转 int, int 转 float, single 转 double, double 转 single。

例子:

```
VCVT.F32.S32 S0, S1    ; signed int -> float  
VCVT.S32.F32 S2, S3    ; float -> signed int
```

3. Move (寄存器移动)

- **VMOV**: 在 ARM general-purpose register 和 FPU register 之间搬运数据。
- 注意: 搬运 bit pattern 不等于完成数值转换。真正数值转换用 VCVT。

易错点: VMOV 是搬运; VCVT 是转换。把整数 bit pattern 搬进 S 寄存器，不代表它自动变成等值 float。

Slide 36 / 原 PPT 41: FPU Instructions - Load/Store 浮点寄存器

这一页讲 FPU 寄存器如何和内存/栈交互。

常见指令：

- **VLDR**: load floating-point register, 从内存加载到 FPU 寄存器。
- **VSTR**: store floating-point register, 把 FPU 寄存器存到内存。
- **VPUSH**: push FPU registers, 把浮点寄存器压栈。
- **VPOP**: pop FPU registers, 从栈恢复浮点寄存器。

示例：

```
VLDR.F32 S0, [R0]      ; S0 = memory[R0]
VSTR.F32 S0, [R1]      ; memory[R1] = S0
VPUSH {S16-S19}        ; save FPU registers
VPOP {S16-S19}         ; restore FPU registers
```

为什么需要 VPUSH/VPOP? 函数调用或中断可能会使用浮点寄存器。如果这些寄存器里的旧值还要用，就必须保存现场，之后恢复。这个逻辑和之前学过的 ‘PUSH R4,LR’ / ‘POP R4,PC’ 一样。

一句话：普通数据用 LDR/STR，浮点寄存器数据用 VLDR/VSTR；普通寄存器压栈用 PUSH/POP，浮点寄存器压栈用 VPUSH/VPOP。

Lecture 18 总结

1. 定点数

- UQm.n: 无符号定点, m 位整数, n 位小数。
- Qm.n: 有符号定点, 1 位符号, m 位整数, n 位小数。
- 真实值: $f = I/2^n$ 。
- 转换: $I = \text{round}(f \times 2^n)$ 。
- 加减: 同格式可以直接整数加减。
- 乘法: 乘积需要右移 n 位。
- 除法: 被除数需要先左移 n 位再除。

2. 浮点数

- IEEE 754 单精度: 1 bit sign + 8 bits exponent + 23 bits fraction。
- 公式: $(-1)^S(1 + \text{Fraction})2^{\text{Exponent}-127}$ 。
- hidden bit: normalized float 默认小数点左边有一个隐藏的 1。
- exponent bias: 单精度 bias=127。
- special values: ± 0 、 $\pm \infty$ 、NaN。
- overflow: 太大; underflow: 太接近 0; subnormal: 用于填补 0 和最小 normalized 数之间的区域。
- rounding: 最近舍入、向 0、向 $+\infty$ 、向 $-\infty$ 。

3. FPU

- FPU 是硬件浮点计算单元。
- S0-S31 是常见单精度浮点寄存器。
- VADD/VSUB/VMUL/VDIV 做浮点算术。
- VCVT 做数值类型转换。
- VMOV 做寄存器搬运。
- VLDR/VSTR、VPUSH/VPOP 负责 FPU 寄存器和内存/栈之间的数据移动。

最后的硬话: 不要把 float 当成“真实小数”。它只是 32 个 bit 按 IEEE 754 规则解释出来的近似值。定点数也不是“小数本身”，只是整数乘上一个固定比例。理解这个，后面所有数值误差问题才不会让你迷路。